

Examen Práctico — Infraestructuras y Procesos de Soporte. Primera convocatoria. Curso 25/26

Grado en Ingeniería del Software · Universidad de Málaga · 12 de junio de 2026

Contexto

Se os proporciona el código fuente de una API REST desarrollada con **Spring Boot 3** y **Java 21** sobre el Mundial de Fútbol 2026. La aplicación expone los siguientes endpoints:

Endpoint	Descripción
GET /	Página de inicio
GET /seleccion/{pais}	Convocatoria de un país con fotos
GET /simulacion	Simulación de la fase eliminatoria
GET /api/jugadores	Lista de jugadores en JSON
GET /api/grupos	Grupos del mundial en JSON

La aplicación lee los datos desde ficheros JSON externos cuya ruta se configura mediante la siguiente variable de entorno:

Variable	Descripción
APP_DATA_PATH	Ruta al directorio <code>data</code> que contiene los ficheros JSON <code>jugadores.json</code> y <code>grupos.json</code>

La estructura del proyecto que se os entrega es la siguiente:

```

├── data/
│   ├── jugadores.json
│   └── grupos.json
├── src/
└── pom.xml
    
```

Ficheros a entregar

Aseguraos de que en la tarea del campus virtual se entreguen los siguientes ficheros:

Fichero	Apartado
Dockerfile	Apartado 1
configmap.yaml	Apartado 2

Fichero	Apartado
<code>deployment.yaml</code>	Apartado 2
<code>service.yaml</code>	Apartado 2
<code>.github/workflows/ci.yaml</code>	Apartado 3

Además, incluid las siguientes **capturas de pantalla** generadas en formato imagen. En cada captura debe verse claramente vuestro nombre de usuario:

1. **Docker — captura de navegador con contenedor en ejecución mostrando la selección española** y del terminal con la salida del comando `docker ps`.
2. **Kubernetes — captura de navegador con contenedor en ejecución en el puerto del servicio mostrando la selección española** y del terminal con la salida de los comandos `kubectl get deploy` y `kubectl get svc`.
3. **Docker Hub — repositorio de la imagen** con todos los tags generados (`main`, `sha-XXXXXXX`, `latest`).
4. **GitHub Actions — Job 1 (`build-and-push`)** con todos los pasos completados en verde.
5. **GitHub Actions — Job 2 (`deploy`)** con todos los pasos completados en verde.
6. **Terminal del self-hosted runner** — captura de la ventana de PowerShell donde se vea el runner ejecutando el workflow (líneas de log con los jobs procesados).
7. **Aplicación actualizada** — captura del navegador en `http://localhost:30080/seleccion/España` donde se vea a **Sergio Ramos** en la convocatoria.

Procedimiento a seguir

(Opción recomendada) Podéis utilizar el template del proyecto pulsando la opción `Use this template`, y crear un nuevo repositorio privado y trabajar directamente desde GitHub. En este caso, podéis crear un nuevo workflow desde GitHub accediendo a Actions → New workflow → Set up a workflow yourself.

(Opción alternativa 1) Si preferís trabajar desde VSCode u otro entorno local, podéis clonar el repositorio y cambiar posteriormente el remoto a vuestro propio proyecto (`git remote set-url origin https://github.com/TU_USUARIO/TU_NUEVO_REPOSITORIO.git`). En ambos casos vuestro repositorio tiene que ser privado. En este caso tendréis que iniciar sesión con Git. Recordad eliminar las credenciales de Git `gh auth logout` y de Windows al finalizar en:

Panel de control → Cuentas de usuario → Administrador de credenciales → Credenciales de Windows.

Paso previo — Carga de imágenes Docker

Antes de comenzar, ejecutad el script `import_images.bat` desde la raíz del proyecto. Este script carga en Docker las imágenes necesarias para el examen, evitando que todos descarguéis a la vez:

```
import_images.bat
```

Al finalizar el script podréis ver en la lista las imágenes `maven:3.9.9-eclipse-temurin-21` y `mundial2026:v1.0`.

Apartado 1 — Construcción del contenedor (3 puntos)

Cread el fichero `Dockerfile` en la raíz del proyecto. El objetivo es que **todo el proceso de compilación y ejecución ocurra dentro del propio contenedor**, sin necesidad de tener Maven ni Java instalados en vuestra máquina.

Utilizad la imagen `maven:3.9.9-eclipse-temurin-21`, que incluye tanto Maven como Java y permite compilar y ejecutar la aplicación en un único paso.

Comandos relevantes dentro del Dockerfile:

- Para compilar el proyecto y generar el JAR (omitiendo los tests):

```
mvn package -DskipTests
```

El JAR resultante se genera en `target/` con el nombre `mundial2026-0.0.1-SNAPSHOT.jar`.

- Para arrancar la aplicación:

```
java -jar target/mundial2026-0.0.1-SNAPSHOT.jar
```

Tened en cuenta lo siguiente al escribir el Dockerfile:

- Copiad el código fuente (`src/`, `pom.xml`) al contenedor antes de compilar. Para aprovechar la **caché de capas de Docker**, copiad primero el `pom.xml` y descargad las dependencias con `mvn dependency:go-offline -B` antes de copiar `src/` — así, si solo cambia el código, Docker reutiliza la capa de dependencias.
- Los ficheros `jugadores.json` y `grupos.json` **no están incluidos en el JAR** — deben copiarse explícitamente en la imagen.
- La variable de entorno `APP_DATA_PATH` debe estar definida apuntando a la ruta donde se copian los JSON dentro del contenedor.
- La aplicación escucha en el puerto `8080`.

Una vez construida la imagen, comprobad que el contenedor arranca correctamente:

```
docker build -t mundial2026:v1.0 .  
docker run -p 8080:8080 mundial2026:v1.0
```

Verificad en el navegador que la aplicación responde en `http://localhost:8080`.

Apartado 2 — Despliegue en Kubernetes (4 puntos)

Utilizando el clúster de Kubernetes integrado en **Docker Desktop**, desplegad la aplicación mediante Kubernetes.

Debéis crear y completar los siguientes ficheros:

k8s/configmap.yaml

Definid un **ConfigMap** llamado **mundial2026-config** que contenga la variable de entorno **APP_DATA_PATH** con el valor adecuado.

k8s/deployment.yaml

Definid un **Deployment** llamado **mundial2026** con las siguientes características:

- Utilizad la imagen construida localmente: **mundial2026:v1.0**
- Inyectad las variables de entorno desde el **ConfigMap**
- La política de descarga de la imagen debe ser **Never**, ya que la imagen es local y no está publicada en ningún registry

k8s/service.yaml

Definid un **Service** de tipo **NodePort** que exponga la aplicación en el puerto **30080**.

Una vez aplicados los manifiestos, comprobad que la aplicación es accesible:

```
kubectl apply -f k8s/
```

Verificad en el navegador que la aplicación responde en **http://localhost:30080**.

Apartado 3 — Actualización continua con GitHub Actions y self-hosted runner (3 puntos)

En este apartado automatizaréis el ciclo completo de build y despliegue mediante un pipeline de GitHub Actions ejecutado en un **self-hosted runner** instalado en vuestra propia máquina.

3.1 — Workflow de GitHub Actions

Cread el fichero **.github/workflows/ci.yml** que se dispare con cada push a la rama **main**. Definid las siguientes variables de entorno a nivel de workflow:

- **IMAGE_NAME:** **mundial2026** — nombre de la imagen
- **REGISTRY:** **docker.io** — registro de Docker Hub

El workflow debe tener **dos jobs**:

Job 1 — **build-and-push**

Este job construye la imagen Docker y la publica en Docker Hub usando las **acciones oficiales de Docker**:

1. **Checkout del código** (`actions/checkout@v4`)
2. **Configurar QEMU** (`docker/setup-qemu-action@v3`) — permite compilar imágenes para múltiples arquitecturas (`linux/amd64`, `linux/arm64`).
3. **Configurar Docker Buildx** (`docker/setup-buildx-action@v3`) — habilita el builder avanzado necesario para la construcción multiplataforma y la caché.
4. **Autenticarse en Docker Hub** (`docker/login-action@v3`) indicando el registro (`REGISTRY`) `docker.io` y usando las credenciales almacenadas como secretos del repositorio:
 - `DOCKERHUB_USERNAME` — vuestro usuario de Docker Hub
 - `DOCKERHUB_TOKEN` — un token de acceso generado en Docker Hub
5. **Extraer metadatos de la imagen** (`docker/metadata-action@v5`) — genera automáticamente las etiquetas de la imagen a partir del contexto del push (rama, tag, SHA corto del commit y `latest` para la rama principal).
6. **Construir y publicar la imagen** (`docker/build-push-action@v5`) con las siguientes opciones relevantes:
 - `context: .` — usa el directorio raíz como contexto de build
 - `push: true` — publica la imagen en Docker Hub
 - `tags` y `labels` — generados por el paso anterior
 - `cache-from/cache-to: type=gha` — reutiliza la caché de GitHub Actions entre ejecuciones para acelerar el build
 - `platforms: linux/amd64` — genera una imagen para procesadores Intel/AMD

Job 2 — `deploy`

Este job se ejecuta **después** de que `build-and-push` termine correctamente en el runner local y actualiza el despliegue en Kubernetes. Debe contener los siguientes pasos:

1. **Actualizar la imagen** del Deployment con la recién publicada:

```
$sha = $env:GITHUB_SHA.Substring(0, 7)
kubectl set image deployment/${{ env.IMAGE_NAME }} ${{
env.IMAGE_NAME }}=${{ env.REGISTRY }}/${{ secrets.DOCKERHUB_USERNAME
}}/${{ env.IMAGE_NAME }}:sha-$sha
```

2. **Esperar a que el rollout termine** antes de continuar:

```
kubectl rollout status deployment/${{ env.IMAGE_NAME }} --
timeout=30s
```

3. **Verificar el estado** del despliegue y los pods:

```
kubectl get deployment ${env.IMAGE_NAME}
kubectl get pods -l app=${env.IMAGE_NAME}
kubectl get events --sort-by='.lastTimestamp' | Select-Object -
Last 10
```

Nota: Para que Kubernetes descargue la nueva imagen al hacer `kubectl set image`, el `imagePullPolicy` del Deployment debe ser `Always` (en lugar de `Never` como en el apartado 2). Actualizad el `deployment.yaml` en consecuencia y volved a aplicar los manifiestos antes de hacer el push.

3.2 — Configuración del self-hosted runner

El workflow usa `runs-on: self-hosted`, lo que significa que los jobs se ejecutan en vuestra propia máquina en lugar de en los servidores de GitHub. Seguid estos pasos para registrar el runner:

1. En vuestro repositorio de GitHub, id a **Settings → Actions → Runners → New self-hosted runner**.
2. Seleccionad **Windows** como sistema operativo. GitHub os mostrará los comandos exactos con el token ya incluido. Los pasos iniciales relacionados con la descarga del archivo ZIP pueden omitirse, ya que este se proporciona previamente: `actions-runner-win-x64-2.3.34.0.zip`. Solo es necesario descomprimir el archivo, abrir una ventana en **PowerShell** en el directorio resultante y ejecutar los siguientes comandos:

```
# Registrar el runner contra vuestro repositorio con todos las
opciones por defecto, excepto la opción de ejecutar como servicio que
se debe de seleccionar No (N)
.\config.cmd --url https://github.com/VUESTRO_USUARIO/mundial2026 --
token <TOKEN_QUE_MUESTRA_GITHUB>

# Arrancarlo
.\run.cmd
```

3. Una vez arrancado, en la página de runners del repositorio aparecerá con estado **Idle** (en espera de trabajos).

Nota: Dejad el runner ejecutándose en una terminal durante todo el examen. Si la cerráis, los jobs quedarán en cola sin ejecutarse.

3.3 — Verificación

Para comprobar que el pipeline actualiza el despliegue correctamente, realizad el siguiente cambio:

El seleccionador español ha realizado un cambio de última hora en la convocatoria. Añadid el jugador **Sergio Ramos** (defensa, dorsal 97) al fichero `data/jugadores.json`.

Haced push a la rama `main` y comprobad que:

- El pipeline de GitHub Actions se ejecuta automáticamente

- La imagen se publica en Docker Hub
 - El despliegue en Kubernetes se actualiza (`kubectl get pods` muestra pods nuevos)
 - Al acceder a <http://localhost:30080/seleccion/España> aparece Sergio Ramos en la convocatoria
-

Notas

- Aseguraos de que el self-hosted runner está activo antes de hacer el push ([Settings](#) → [Actions](#) → [Runners](#)).
- Maven no es necesario en la máquina al estar incluido en el Dockerfile.